

Resilient Event Dispatcher System

In decoupled microservices architectures, components communicate via asynchronous event streams rather than direct procedural calls. A centralized "Event Dispatcher" orchestrates this flow, allowing specialized modules to subscribe to specific event types and react to broadcasted payloads without tight coupling.

Implement an `EventDispatcher` class that manages complex event-driven workflows. The system must support multi-subscriber registration, handle instance-specific unsubscriptions, and ensure **fault tolerance**: if a subscriber callback fails, the dispatcher must catch the exception, log the error, and proceed to notify remaining subscribers.

Example 1 Input:

```
SUBSCRIBE user_login log_event SUBSCRIBE user_login process_data  
DISPATCH user_login {"id": 101, "message": "Welcome"}
```

Output:

```
Log: Welcome Process: 101
```

Explanation: Multiple callbacks (`log_event`, `process_data`) are registered for the `user_login` event. When the event is dispatched, the dispatcher executes both callbacks in the order of subscription, passing the dictionary payload.

Example 2 Input:

```
SUBSCRIBE data_sync process_data DISPATCH data_sync {"fail": true}
```

Output:

```
Error in process_data: Processing failed
```

Explanation: The `process_data` callback encounters a critical error. The resilient dispatcher catches the exception and prints a standardized error message including the function's name (`__name__`), ensuring the system remains stable.

Function Parameter:

- `event_type` (str): The unique identifier for the event category.
- `callback` (callable): The handler function to be executed.
- `data` (dict): The message payload.

Returns

- `None`: Operations are stateful within the dispatcher instance.

Constraints

- Maintain strict subscription order during dispatching.
- Catch all internal callback exceptions and print: `Error in <callback_name>: <exception_message>`.
- Handle duplicate subscriptions by executing the same callback multiple times.

Input Format for Custom Testing Input from stdin will be processed as follows: Each line contains a command: `SUBSCRIBE <event> <callback_id>`, `UNSUBSCRIBE <event> <callback_id>`, or `DISPATCH <event> <json_data>`.

Sample Case 0 Sample Input

```
SUBSCRIBE auth_fail log_event DISPATCH auth_fail {"message": "Invalid password"}
```

Sample Output

```
Log: Invalid password
```

Explanation The `log_event` callback is successfully triggered by the `auth_fail` dispatch.